Computer Security - CheatSheet

IN BA5 - Thomas Bourgeat

Notes by Ali EL AZDI

October 26th, 2025

CompSec Properties - **Confidentiality**. prevent unauthorized disclosure of information. *authorized users may read a file*
- **Integrity.** prevent unauthorized modification of information. *authorized programs may write a file*
- **Availability.** prevent unauthorized denial of service or access to information and resources. *authorized services can access a file*
- **Authenticity.** assurance that entities (users, systems, or data) are genuine and can be verified as such.
- **Anonymity.** protection of an individual's identity from being disclosed or linked to specific actions or data.
- **Non-repudiation.** assurance that a party in a communication cannot deny the authenticity of their signature or the sending of a message.

The Adversary. malicious entity aiming at breaching the security policy and **will** choose the optimal way to use its ressources to mount an attack that violates the security properties.
Threat Model. describes the ressources available to the adversary and their capabilities *(has access to internet, but doesn't have access to the internal network of the company.)*
Threat. Who might attack which assets, using what resources, with what goal, how, and with what probability
Vulnerability. Specific weakness that adversaries could exploit with interest in a lot of assets *(API is not protected, password appears in plain text...)*
Harm. The bad thing that could happen when the **threat** materializes. *(adversary steals the money, learns my password...)*
Security Policy. high level description of the security properties that must hold in the system in relation to assets and principals.
- **Assets (objects).** anything with value (data, files, memory) needing protection.
- **Principals (subjects).** people, computer programs, services.

Security Mechanism. Technical mechanism used to ensure that the security policy is not violated by an adversary within the threat model, **we can only prepare for threats we're aware of**
(Policy. ensure messages cannot be read by anyone but the sender and the receiver, Mechanism. encrypt the message before sending)
Composition of Security Mechanisms
- **Defence in depth.** As long as one remains unbroken the Security Policy isn't broken) *(two-factor auth)*
- **Weakest Link.** if anyone fails the Security Policy, it is broken. *(security questions for a lost password → just need to know the answer...)*

Humans can be vulnerabilities - phishing attacks, bad use of passwords...)
To show a system is secure. (under a *specific* threat model)
Attacker - Just one way to violate **one** security property is enough.
Defender - No adversary strategy can violate the security policy.
Security Argument. Rigorous argument that security mechanisms in place are effective in maintaining security policy subject to assumptions on Threat Model.

Principles of CompSec.
1. **Economy of mechanism** Keep security mechanism design simple and small ⇒ Easier to audit and verify, testing is **not** appropriate to evaluate security.
- **Trusted Computing Base (TCB).** Every component of the system on which the security policy relies upon *hardware, firmware, software.*

The TCB is trusted to operate correctly for the security policy to hold. → If something goes wrong in it, the security policy may be violated
must be kept small to easy verification / diminish the attack surface
2. **Fail-safe defaults.** Base access decisions on permission rather than exclusion. *(Whitelist, do not blacklist)*

If something fails, be as secure as it does not fail errors / uncertainty should error on the side of the security policy.
3. **Complete mediation.**. **Every** access to every object must be checked for authority. A **Reference Monitor** mediates all actions from subjects on objects and ensures they are according to the policy. ⚠ *time to check vs. time to use*
4. **Open design** The design should not be secret *Always design as if the enemy knows the system. (Crypto-only secret is key.Authentication-Only secret is password.for Obfuscation-Only used secret is noise.).***assuming the thread model can't get a hold of the system is unrealistic (employee corruption, ...)**
5. **Separation of privilege.** No single accident, deception, or breach of trust is sufficient to compromise the protected information
- **Privilege.** A privilege allows a user to perform an action on a computer system that may have security consequences. *(create a file in a directory...)*

6. **Least Privilege.** Every program and every user of the system should operate using the least set of privileges necessary to complete the job. *Rights are added on need, discarded after use. Users should only know about things if they **have** to.*
7. **Least Common Mechanism** Minimize the amount of mechanism common to more than one user and depended on by all users. Every shared mechanism represents a potential information path between users.
8. **Psychological acceptability.** It is essential that the human interface be designed for ease of use, so that users routinely and automatically apply the protection mechanisms correctly. *(hide complexity, cultural acceptability...)*
9. **Work Factor**

Compare the cost of breaking the mechanism with the resources of a potential attacker. *(cost of compromising insiders, cost of finding a bug, monetization...)*
10. **Compromise recording** Detect and record security breaches with tamper-evident logs, ensuring traceability, integrity, confidentiality, and availability of recorded data.

Access Control. Security mechanism that ensures all accesses and actions on objects by principals are within the security policy.
no *chicken soup (checks everywhere in code), use a reference monitor, module used all over code checking for **subjects** and **actions***
Discretionary Access Control (DAC). Object owners assign permissions, ownership of resources. *Linux, Social Networks*
Mandatory Access Control (MAC). Central security policy assigns permissions, usually for organizations with need for central control. *Military, Hospital,....*
Access Control Matrix. abstract representation of all permitted triplets of (subject, object, access right) within a system.

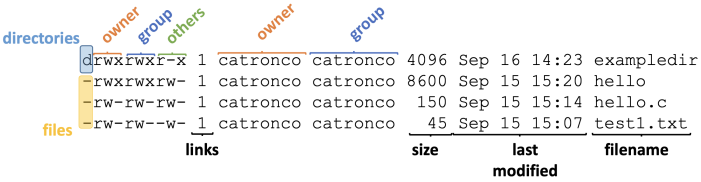
	file1	**file2**	**file3**
Alice	read, write		read
Bob		read, write	read, write

Complexity. $O(f \cdot u)$
Access Control List (ACL). associate permissions to objects, stores permissions close to the resource.
file1: (Alice,read/write), file2: (Bob, read/write),
file3: (Alice,read),(Bob,read/write)
⊕ easy to determine who can access a resource and to revoke rights by resource.
⊖ hard to check all users rights, to remove all perms. for a user, delegate perms.
Role Based Access Control (RBAC). access granted based on user roles and predefined permissions. systems have too many subjects (that come and go) → large dynamic ACLs. Subjects are often similar to each other and get assigned the same rights.
1. Assign permissions to roles, 2. Assign roles to subjects
3. Subjects have the permissions of their assigned role

⊖ role explosion (temptation to create fine grained roles), limited expressiveness, difficult to implement separation of privilege + least privilege.
Group Based Access Control (GBAC). access granted based on user group membership. **exactly like RBAC but instead of roles, permissions are assigned to groups.** groups are broader, represent organizational units instead than specific functions.
1. Assign permissions to groups 2. Assign subjects to groups
3. Subjects have the combined permissions of all their groups

⊖ coarse granularity, overlapping group memberships, inconsistent permissions, difficult to manage if users belong to many groups.
*In case of **Negative Permissions** check negative permissions first before group, ⚠ system crashes before negative checks.*
Ambient Authority. an action succeeds if the subject only specifies the *operation* and the *object name*, not the specific authority used.
⊖ leads to accidental misuse of authority, programs may act with more rights than intended. → Confused Deputy Problem.
Confused Deputy Problem. when a program (deputy) is tricked into misusing its authority on behalf of another subject.
Alice runs compiler(input, bill) ⇒ compiler (with write access to bill) overwrites billing file. ⇒ Alice uses compiler's authority to modify bill indirectly.
⊖ ambient authority allows unintended privilege use.
Solutions:
1. Restrict privileged process access.
2. Make privileged process check user's authorization.
3. Use **Capabilities** to explicitly delegate rights.

User & Group Identities. Most modern systems rely on DAC.
UIDs / GIDs. numerical identifiers for users and groups.
/etc/passwd: `username:password:UID:GID:info:home:shell`
/etc/group: defines secondary groups.
Each user has a home directory and belongs to one or more groups.
UNIX Model. Everything is a file.



Directories. Read → list files; Write → add/remove files; Exec → traverse (cd).
Permission check order: 1. If process UID = file owner → check owner bits.
2. Else if GID matches → check group bits.
3. Else → check "other" bits.

Root (UID 0. bypasses all checks.
Changing Permissions.
chmod. modify permission bits. `chmod +r file`, `chmod 666 file`.
chown. change file owner/group(opt.) `chown root:staff /srv/config`
chgrp. change file group. `chgrp www-data /var/www`
Special Rights.
suid (set user ID). run program with owner's privileges. *puts s in owner's x field*
`chmod u+s filename`
allows normal users to change passwords without full root access.
sgid (set group ID). run program with group's privileges.
on a directory - creating a file inside gets folder's group
`d rwx r-s r-x 2 root staff /srv/shared`
sticky bit. on directories, prevents deleting/renaming files you don't own.
`d rwx rwx rwt 10 root tmp /tmp`
Special Users.
root: UID 0, full privileges, bypasses checks, in TCB.
`sudo` run as root `su [username]` switch to account(default is root)
nobody: UID -2, owns no files, minimal privileges, safer for untrusted code.(least privilege)
⊕ flexible, simple model, widely adopted.
⊖ relies on **ambient authority**, prone to Confused Deputy attacks.

Windows: DACL. Controls access to objects via list of ACEs.
ACE (Acces Control Entry). `<Type, Principal, Permissions, Flags>`.
- **Types.** `Allow / Deny` (negative / positive). Deny takes precedence and ordered with Denied permissions first.
- **Principal.** User or group (SID).
- **Permissions.** Fine-grained rights *(Read, Write, Execute, Delete...)*
- **Flags.** Inheritance, propagation, object-specific.

Access Tokens. Thread/process carries user + group SIDs checked against DACL.
Least Privilege. Users run limited by default; elevation via "Run as admin."

Capabilities. associate permissions to *subjects*
Alice: {(file1, read/write)} Bob: {(file2, read/write), (file3, read/write)}
⊕ easy to determine all permissions of a user and to delegate rights by subject.
⊖ hard to determine who can access a resource or to revoke rights by resource.

Security Model. a design pattern for a set of properties. (⚠not covered by model - who are the subjects? what are the objects? what mechanisms to use to implement it?)

Bell-La Padula (BLP).
security model where Subjects S and objects O are associated to a level of **Confidentiality**.
Access rights. Execute, Read, Append, Write.
- Objects are associated to a Security Level = (Classification, set of categories).
{(Unclassified < Confidentiality < Secret < Top Secret), {NATO, Crypto, Nuclear}}
Dominance Relationship.
Transitive, There always is a Top and Bottom,
Only partial ordering (some pairs of elements can't be compared).
A security level $(l_1, c_1$. dominates $(l_2, c_2$. if and only if $l_2 \leq l_1$ and $c_2 \subseteq c_1$.
eg.$(Secret, \{Nuclear, Army\}) \geq (Confidential, \{Army\})$
Clearance. max security level a subject has been assigned. *clearance-level(S_i).*
Current Security level. subjects can operate at lower security levels. *current-level(S_i).*
Declassification. Removing classification labels from information so it can be shared with lower security levels. *soldier (S) with Secret clearance shares sanitized info like "We've seen tanks moving" with a lower-level user (U).*

BLP to create Confidentiality Policies.
Simple Security Property.
if (subject, object, w/r) is a current access, $\Longrightarrow$ level(subject) dominates level(object).
(SUBJECTS CAN'T READ UP)

Star Property. If a subject has simultaneous "observe" (r,w) access O_1 and "alter" (a, w) access to O_2 then level O_2 dominates level O_1.
(SUBJECTS CAN'T WRITE DOWN). *changing object's perms is write-like.*

Discretionary Property. A subject may only access an object if it has explicit permission for that specific type of access (r, w, x, etc.) defined by the access control matrix.

Basic Security Theorem(induction). if all state transitions are secure, and the initial state is secure, then every subsequent state is secure regardless of the input.

Covert Channels. Any channel that allows information flows contrary to the security policy, *Storage channels(shared counters, ...), Timing channels(queuing time...).*
⚠Least Common Mechanism. *mitigated by adding noise or isolation(no high ↔ low level communication).*

BIBA (Integrity).
Security model where subjects S and objects O are associated to a level of **Integrity** (trustworthiness).
Access rights. Read (Observe), Write (Modify)
BIBA to create Integrity Policies.
Simple Integrity Property.
If (s, o, r) is a current access, then level(s) dominates level(o).
(SUBJECTS CAN'T READ DOWN) $\Rightarrow$ prevents contamination from low to high.

$*$**-Integrity Property.**
If (s, o, w) is a current access, then level(o) *does not* dominate level(s).
(SUBJECTS CAN'T WRITE UP) $\Rightarrow$ prevents low from corrupting high state.

Discretionary Property (DAC). Subject must still have explicit permission for the access (r, w, x) per the access control matrix.

Basic Integrity Theorem (induction). If the initial state is integrity-secure and all state transitions satisfy BIBA properties, then all reachable states preserve integrity.

BIBA Low-Water-Mark Variants (taint-tracking).
1. *For Subjects.*
Subjects start at their max integrity, **on read**:
current-level(s) = min(current-level(s), level(o)).
$\Rightarrow$ once tainted, subject sinks; prevents writing up thereafter.
2. *For Objects.* **On write** by s:
level(o) = min(level(o), level(s)).
$\Rightarrow$ objects "sink" easily; can cause integrity collapse.

Mitigation: replicate high/low objects; sanitize before promotion; detect/flag unexpected level drops.

BIBA Addtional Actions - Invoke.
Simple Invocation. only allow subjects to invoke subjects with a label they dominate.
⊕ protect high integrity data from misues by low integrity principals
⊖ what level is the output ?
Controlled Invocation. Only allow subjects to invoke subjects that dominate them.
⊕ prevents corruption of high integrity data
⊖ hard to detect polluting information.

Sanitization (lifting low → high).
- *Fail-safe default:* deny by default; elevate only after checks pass.
- *Whitelist over blacklist:* validate that *all* required properties of "good" hold.
- *Context-aware:* encoding, schema, range, semantics, and provenance checks.
Note: Sanitization bugs commonly break integrity guarantees.
Principles for Integrity.
- **Seperation of Duties.** Require multiple principals to perform an operation
- **Rotation of Duties.** Allow a principal only a limited time on any particular role and limit other actions while in this role
- **Secure Logging.** Tamper evident log to recover from integrity failures. Consistency of log across multiple entities is key.

Chinese Wall Model. All objects are associated with a label denoting their origin.
(Pepsi, Coca-Cola, Microsoft Audit, Microsoft Investments)
Define **conflict sets** of labels. {*Pepsi, Coca-Cola*}, {*Microsoft Audit, Microsoft Investments*}.
Subjects are associated with a **history** of their accesses to objects and their labels.
Access Rules. A subject can access an object (read or write) **only if** the access does not allow information flow between items with labels in the same conflict set.
Example (Direct Flow).
1. Access to Pepsi (*OK*)
2. Access to Microsoft Invest (*OK*)
3. Access to Coca-Cola (*Denied*)
Example (Indirect Flow).
1. Alice accesses Pepsi
2. Bob accesses Coca-Cola and IBM
3. Alice tries to access IBM (*Denied.indirect link via Bob*)
Sanitization.
Allows more flexibility by "un-labeling" some items→controlled sharing/reuse of data.

All together (Asymmetric Cryptography) - Bob sends a message to Alice
1. Get public keys (via PKI).
Bob retrieves Alice's encryption public key PK_{Alice} and verification public key PK_{Alice}.
2. Prepare the message.
Bob has message M and computes its hash $h(M)$.
Allows: the signature to cover the exact content of M while keeping computation efficient.
3. Sign the message hash.
Bob uses his secret signing key SK_{Bob} to generate a digital signature. $\text{Sign}_{SK_{\text{Bob}}}(h(M))$.
4. Encrypt the message.
Bob encrypts the message using Alice's encryption public key PK_{Alice}: $E_{PK_{\text{Alice}}}(M)$.
Allows: only Alice, who holds the matching secret key SK_{Alice}, to read the message.
5. Send encrypted message and signature.
Bob sends $E_{PK_{\text{Alice}}}(M), \text{Sign}_{SK_{\text{Bob}}}(h(M))$ to Alice.
Allows: transmission over untrusted channels while maintaining confidentiality and authenticity.
6. Decrypt the message
Alice uses her secret encryption key SK_{Alice} to decrypt. $D_{SK_{\text{Alice}}}(E_{PK_{\text{Alice}}}(M)) = M$.
7. Verify the signature.
Alice computes $h(M)$ and verifies Bob's signature with his verification public key PK_{Bob}:
$V_{PK_{\text{Bob}}}(M, S) =$ True if valid.
Allows: Alice to confirm the message was indeed sent by Bob and was not modified.
Authenticity. only sender can generate a valid signature, allowing the server to verify the sender's identity and preventing impersonation. **Integrity.** Any message change invalidates signature, ensuring tampering is detectable. **Non-repudiation.** only sender can produce the valid signature → they can't deny later having sent the message.

Block Cipher - Electronic Code Block (ECB).
encrypt each message block independently using the same key.
1. The message M is divided into blocks.
$M = M_1 M_2 M_3 M_4$.
2. Each block is encrypted separately $C_i = E_K(M_i)$.
3. Decryption. $M_i = D_K(C_i)$.
⚠If $(M_i = M_j) \Rightarrow (C_i = C_j) \Rightarrow$ (freq. analysis)

Block Cipher - Cipher Block Chaining (CBC)
add randomness/inter-block dependence by chaining blocks.
$C_0 =$ IV **Encryption.** $C_i = E_K(M_i \oplus C_{i-1})$
Decryption. $M_i = D_K(C_i) \oplus C_{i-1}$
IV must be random / isn't a ciphertext block but needed for decryption.
- If IV incorrect, only first plaintext block will decrypt incorrectly, later blocks remain unaffected.

Cryptography.
Data in transit. Securing communications. **Data at rest.** Securing stored informations
Let C the Ciphertext, K the Key, M the Message/Plaintext. $C = E_K(M)$ and $M = D_K(C)$
Keyspace. Number of possible keys that can be used in an encryption algorithm.
Invertibility Requirment. $\forall K, M | D_K(E_K(M)) = M$ (otherwise can't recover message)
Security Requirement. Functions should be hard to invert without knowing K.
Ideal Case - adversary must try every possible combination of keys (bruteforce).

Caesar's Cipher
- Encryption. Shift each letter by a fixed number (K)
- Decryption. Shift each letter by a fixed number (-K)
Keyspace. Only 25 possible keys.
$\log_2(25. = 4.6$ bits of security (too small).
Both can be broken using **Frequency Analysis Attack**.

The Substitution Cipher message
Each Letter is mapped to a unique, different letter, defined by a permutation of a 26 letters alphabet.
Keyspace. $26! \approxeq 4.03 * 10^{26}$

Frequency Analysis. Use statistical properties of the language.
1. Most frequent letter in English is 'e'. 2. Identify most frequent letter in Ciphertext. 3. Map it to 'e'.
Ideally, An N-bit key should offer security as close to N bits as possible (require 2^n attempts), if not the algorithm is considered **broken**.
Types of Adversaries. security models
- Passive Eavesdropper - adversary can only read the ciphertext
- Active Attacker - adversary can influence the system *(corruption of one of parties, ...)*

Known Plaintext Attack (KPA).
Active security model. Attacker gets access to some pairs, (message, ciphertext), all encrypted with secret K.
From these pairs, she tries to guess key
K to decrypt other messages
realistic because of message headers, message headers ("From:...", timestamps...)

Chosen Plaintext Attack (CPA).
- Suppose Eve convinces Alice to encrypt chosen messages with the secret key
- For all m, Eve gets access to $E_K(m)$. Eve has access to an Encryption Oracle.
(realistic, she could get access to an encrypted messaging app for limited time, or to an encryption api).
⚠*if Eve encrypts entire alphahabet she can reveal the key instantly.*

Side-Channels Attacks
- Timing attack - Eve measure how long it takes for a given message to get encrypted or a ciphertext to be decrypted
- Power Analysis - Eve observes the energy consumed by the device doing the crypto.

One-Time-Pad (OTP).
Goal - Remove frequency analysis
- Use a key of random bits as along as the message.
- $Enc(k, m) = m \oplus k$
- $Dec(k, m) = Enc(k, m) \oplus k$
Key should be random for every sent message. Otherwise attacker can collect information about the message.
Integrity Attack Example (Eve flips the first bit of M):
1. Eve flips the first of C to get C'
2. Bob decrypts C': $M' = C' \oplus K = (M \oplus K \oplus \Delta) \oplus K = M \oplus \Delta$

OTP is **Perfectly Secure**.
- For any ciphertext C, every possible plaintext M is equally likely
- Key K is uniformly random, so C gives the adversary zero new information about M.
Perfect Secrecy.
$\forall m, c, P(M = m | E_k(m) = c) = P(M = m)$.
Guarantees **confidentiality**
⊖ message-length/used once keys.
Reuse of key Attack. Send two messages of length 5 with key reuse.
$C_1 \oplus C_2 = M_1 \oplus M_2$

Symmetric Cryptography Schemes
Encryption/decryption done with the **same key**
- **Block Ciphers.** Operate on fixed-size blocks.
- **Stream Ciphers.** Operate on bit/byte at a time, like pseudo-OTP.

Symmetric Cryptographic key
-**Known to both parties.** Partners must agree on the key before starting using the primitive
-**Reused.** The keys is pre-shared once and then reused *(but keys have a "duration")*
-**Must be secret.** Reveling the key eliminates any protection provided by the primitive

Stream Ciphers - The pseudo-OTP. emulate OTP while solving key-length problem.
1. Secret short Key (K) shared between Alice and Bob **2.** Key Stream Generator - Uses K and Initialization Vector (IV) → an arbitrary long, pseudo-random bit stream (S). **3.** Encryption. $C = M \oplus S$
(generator needs a key as main seed to generate a predictable random sequence, an iv for the sequence to start differently on each run)
⊕ Speed, Low Error propagation(errors in one bit do not affect subsequent symbols)
⊖ Low Diffusion (a change in a bit only affects one bit), Susceptibility to Modification (low diffusion makes it easier to tamper)
⊖ key stream generators are periodic because seed is finite. Thus, we need period long enough to not an issue (avoiding frequency analysis)

Linear Feedback Shift Register for Key Stream Generators. build a """random""" sequence of bits using a linear recurrence relation on a sequence of bit.
Example: Starting with state $a_0, a_1, a_2, a_3 | a_n = a_{n-3} \oplus a_{n-4}$ (⊕ Easy to build/analyze.)
Randomness of LFSR.
if characteristic polynomial of the recurrence relation of the LFSR is primitive. The maximum possible number of states before repeating. For an L-bit register, the maximum period is $2^L - 1$ states
Distribution property.
As a consequence, the sequence generated exhibit good distribution properties. Every possible non-zero state appears exactly once in a cycle.
⊖ underlying operation is **linear**. *(some algorithms can recover this with a stream subsequence)*

Symmetric ciphers are fast, but require a **pre-shared secret key**
Diffie-Hellman Setup. Alice, Bob (and Eve) know large public parameters:prime p/generator g

Alice's Actions	*Bob's Actions*
1. Chooses **private** secret a.	1. Chooses **private** secret b.
2. Computes Public Value $A = g^a \bmod p$	2. Computes Public Value $B = g^b \pmod p$
3. Sends A to Bob.	3. Sends B to Alice

Eve sees A, B, g, and p, but can't find a/b due to the *Distrete Log Problem.*
Both Alice and Bob can compute a shared secret K: $K = B^a \pmod p = (g^b)^a \bmod p = A^b \pmod p$
this is uses **Trapdoor functions**, easy to compute but hard to revert.

Man-In-The-Middle (MITM).
Vanilla DH is Vulnerable to Active Attack,
DH guarantees key agreement, **not** authenticity.
Attacker could intercept A from Alice and B from Bob, and then impersonate.
1. Eve sends E_B to Alice. Alice computes K_{AE}
2. Eve sends E_A. Bob computes K_{BE}
3. Creates an Alice-Eve channel and Eve-Bob channel
she can read, modify, and relay all communication.

Asymetric Cryptography. Encryption using a public–private key pair. Public key can be stored on a public server, the **Public Key Infrastructure (PKI)**.
Hash Functions. maps any-length message to a fixed-size output using a hash function. $h(M) = H$
Security properties. 1. **Pre-image resistance.** Given $H = h(M)$, it is hard to find M.
2. **Second pre-image resistance.** Given M, it is hard to find another $M' \neq M$ such that $h(M') = h(M)$.
3. **Collision resistance.** It is hard to find any two values M and M' such that $h(M) = h(M')$.

Block Cipher - Counter Mode
add randomness per-block without inter-block dependencies.
(parallel)
Encrypt. $C_i = M_i \oplus E_K(\text{Nonce} + i)$.
Decrypt. $M_i = C_i \oplus E_K(\text{Nonce} + i)$.
Nonce must be unique (never with same key) reuse leaks keystream.

Block Cipher - CBC-MAC. turns block cipher into MAC by chaining message blocks with encryption.
$C_0 = IV$ **Encrypt.** $C_i = E_K(M_i \oplus C_{i-1})$ $\text{MAC}_K(M_1, \ldots, M_n) = C_n$
deterministic. fixed IV → same message/same MAC.
⚠message length must be known and fixed; otherwise, extension attacks are possible (e.g., using $M|(T \oplus M')$).

Message Authentication Code (MAC). short tag computed from a message and secret key(pre-shared) to verify message **integrity** and **authenticity**.
1. Encrypt-and-MAC. send $E_K(P_1P_2)$ and $MAC_K(P_1P_2)$. integrity of plaintext but no integrity on ciphertext/may reveal info on ciphertext. **2. MAC-then-Encrypt** send $E_K(P_1P_2||MAC_K(P_1P_2))$. hides MAC and provides confidentiality/integrity on plaintext but no integrity ciphertext.
3. Encrypt-then-MAC send $E_K(P_1P_2)||MAC_K(E_K(P_1P_2))$. **authenticates** ciphertext, ensures **confidentiality/integrity** ✓

Authentication. the process of verifying a claimed identity. (Prove who you are.)
Passwords. secret shared between user and system.
Problems:

I **Secure transfer.** password may be eavesdropped when communicated.
Encrypting makes password unreadable to eavesdropper, but still allows copy and send.
(⚠**Replay-Attacks.** sending same request without needing the password)
Challenge-Response protocol solves this.
1. Principal declares they want to login.
2. System authenticating principal sends a **Challenge** (a random value).
3. Principal encrypts password *together* with the challenge.(ensures password is never encrypted with same value and encryption is fresh)
4. After checking password, System *must* delete the challenge.

II **Secure check.** naïve checks may leak information about the password.
Check Options:
1. Letter by letter - Time to check leaks the number of correct checked letters. (Timing channel)
2. Check everything - Same time regardless of input (hashing solves this by default).

III **Secure storage.** if stolen entire system is compromised.
Passwords must be stored to be checked against but *not stored in clear* → an adversary can read them and impersonate users.
Storage Options :
1. Encrypted Passwords. requires key to be stored in same system as database, not secure.
2. Hashed Passwords. Not a solution, hashing does **not** require a key. ⚠passwords are not truly random.→Database can be downloaded and compared against hash of most common passwords (**Dictionary Attack**).
3. Hashed + Salted Passwords (Do this !). **Salt.** unique random value generated per password.
We store: (username, $H('123456'||\text{salt}_n)$, salt_n).
Dictionary attacks are still possible but same passwords are undetectable.
Principal asks to login→*System* sends Challenge→*Principal* sends encrypted password with Challenge →*System* checks Challenge, decrypts, looks up hash by username, hashes plaintext with salt and checks if same as stored.

IV **Secure passwords.** easy-to-remember passwords tend to be easy to guess
Biometrics. measurement and statistical analysis of people's unique physical characteristics(fingerprint, face, …).

Biometrics Authentication.
1. **Enrollment.** biometric is captured by a sensor and processed to be converted in a stream of bits(Biometric Template). **Capture → Process → Store**.
2. **Verification.** biometric is captured, processed and compared against the stored one. **Capture → Process → Compare**. (threshold for match vs. no match).
Balance between False Positives (wrongly authenticated) and False Negatives(wrongly rejected).
Storing / Processing can happen locally(⊕ privacy) / remotely(⊕ security).

Authentication Tokens. by proving ownership of a token. (Smart Card, …)
1. Offline - Device and Server establish a common *seed* (random value) & synchronize their clocks.
2. When Device wants identity:
(a) Server asks to prove it. (b) They both compute $n = \frac{now-start}{interval}$. *Current interval.*
(c) Device applies **keyed** function (pre-shared key) n times on initial seed. $v = f^n(seed)$
(d) Device sends result to Server. (encrypted, eavesdropper can't have the seed).
(e) Server computes same value and checks against the one received.

Can't be a hashing function (hash functions aren't keyed!, eavesdropper could compute $h_k(seed)$ for any k after the one observed.).

Security Engineering process.
1.Define security policy *(principals, assets, properties)* and a threat model. 2.Define security mechanisms that support policy of threat model. 3.Build implementation to support the mechanisms

Attack Engineering process.
exploit weaknesses introduced during the security engineering process due to lack of careness/knowledge or errors.

Threat Modelling. process to identify threats / unprotected resources.
- **Attack Trees.** represent goal as the root and ways to achieve it as branches.
- **STRIDE.** 1) Identify assets, principals, trust boundaries, and named flows **2) For each threat:**
1. STRIDE category, 2.what changes/abuse happens, 3.which flow it affects, 4.concrete mitigation.
Spoofing: fake identity (user/service). **Tampering:** unauthorized edit of data/messages/code.
Repudiation: deny action (no provable record). **Info disclosure:** leak confidential data/metadata.
Denial of service: exhaust/block resources/reduce availability. **Elevation of privilege:** gain higher privileges than allowed.
- **PASTA.** start from buisness goals, processes, and use cases. Find threats in buisness model, assess impact, prioritize on risk.

Methods.
- **GET.** requests a resource; parameters are appended to the URL as query key–value pairs after "?", separated by '&'.

`http://www.mywebsite.com/apparel/skirt.php?sku=123&lang=en&sect=silk`

protocol | hostname / domain | directory | filename | query parameters

- **POST.** creates or updates a resource; sends data in the request body (not the URL) and typically modifies server data.

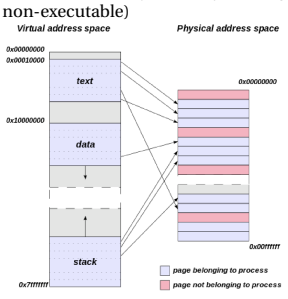
```
POST /test/demo_form.php HTTP/1.1
Host:  w3schools.com
name1=value1&name2=value2
```

Cookies. small piece of data stored by browser on client (creates HTTP "sessions"). *also used to track users.*. **Ambient Authority** is used with cookies. *logged into bank.com → cookies stored for bank.com.* ***Any*** *(!CSRF) new HTTP requests to this website* ***will include all cookies*** *to continue session.*

URL. standardized address that specifies a resource's access method, host, and path.
HTML. markup language using tags to structure content and tell browser how to render a webpage.
WEP. Wi-Fi security protocol that encrypts frames with **RC4** (stream cipher with random 24-bit IV). IV repeats frequently, causing keystream reuse and enabling message (and sometimes key) recovery.
PHP. server-side scripting language for dynamic web pages; uses variables and request/session data ('$_GET', '$_POST', '$_SESSION') and outputs HTML with 'echo'.
Lifetime of a GET/POST request.
1. Browser sends HTTP GET/POST request.
2. Web Server receives requests and identifies if it's a static asset(.jpg,…) or dynamic(.php).
3. PHP interpreter executes called .php 4. Sends response to web browser

---

Data Execution Prevention (DEP). Mark memory pages as writable OR executable, never both. (prevents code injection by making stack/heap non-executable)



⊕ low-overhead/hardware enforced
⊖ no self-modifying code.

Code Reuse - Control Flow Hijack. Bypass DEP by chaining existing executable code (gadgets) instead of injecting new code.
Attack Chain:
1. **Memory Safety Violation.** `strcpy` writes past `tmp[MAX]` (spatial error)
2. **Integrity (*C).** Attacker overwrites return address and adjacent stack data
3. **Location (&C).** Return address points to `&system()`, stack contains `&"/bin/sh"`
4. **Usage (*&C).** CPU dereferences corrupted pointer, jumps to `system()`
5. **Gadget Chaining.** Existing code (`system()`) executes with attacker-controlled arguments, no shellcode needed

---

Common Weaknesses Enumeration (CWE). database of software errors leading to vulnerabilities.
1. **CWE 1 - Insecure Interaction Between Components.** unchecked information sent between different components.
(a) **CWE 78 - OS Command Injection.** application embeds unvalidated user input into an OS command, allowing attackers to execute arbitrary commands *ls /home/user*.
(b) **CWE 79 - Cross Site Scripting.** application embeds unvalidated user input into dynamically generated web content, causes browser to execute injected scripts.
Common Script Injections.

```
<img src=x onerror=alert(1)>;
<svg onload=alert(1)>;
<body onload=alert(1)>;
<a href=javascript:alert(1)>text</a>;
```

Useful Functions.
`navigator.userAgent`
Returns the browser identification string; increases fingerprinting surface.
Chrome/143.0.0.0 Mobile Safari/537.36
`window.location.replace(url)`
Forces navigation to `url` without adding a history entry.
`document.cookie`
Returns all non-`HttpOnly` cookies for the current origin; exposed session/state cookies immediate account compromise on XSS.
`document.domain`
Gets/sets legacy origin relaxation; can broaden trust across subdomains and weaken same-origin isolation.
`document.referrer`
Returns the referrer URL when sent; can leak sensitive paths/parameters without strict Referrer-Policy.
`document.URL`
Returns the full current document URL

(c) **CWE 362 - Cross Site Request Forgery.** malicious website makes a state-changing action using user's session cookie for another website.

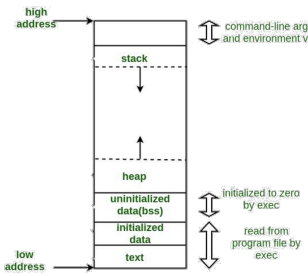
> **Same Origin Policy (SOP).** web browser security mechanism that restricts scripts from one origin from reading data from another origin. An origin is a (protocol, host, port) *http:example.com.*
> *This doesn't prevent CSRF as it's based on sending a request and that the browser naturally attaches cookies of the destination domain in request (unless special setting).*

Solutions:
For every state-changing endpoint:
- Check Origin header and only allow exact origin.
- If Origin missing, require Referer and ensure it starts with expected origin, else reject.
Keep GET side-effect free; Require a CSRF token (challenge) on every state-changing request; For high-riks actions require re-auth.

2. **CWE 2 - Risky Ressource Management.** not properly managed creation, usage, transfer, or destruction of important system resources
3. **CWE 3 - Porous Defenses.** defensive techniques misused, abused, or just ignored

---

Software Security.
Stack. Call frames: locals, args, return addresses; grows down.
Heap. Dynamic allocations via `malloc/new`; grows up.
Uninitialized data (BSS). Global/static zero-initialized storage; zeroed at program start.
Initialized data. Global/static with explicit initial values; loaded from the executable.
Text. Executable code (often read-only); mapped from the executable.
Args & env. `argv`/environment strings placed at startup near the initial stack.



Memory Corruption.
Unintended modification of memory location due to missing / faulty safety checks.

```
void vuln(int user1, int*array){
  // no bound check for user1
  array[user1] = 42;}
```

Memory Safety - Temporal Error. tried access to memory initially reserved for our program but not reserved anymore.

```
void vulnerable(char* buf) {
  free(buf); // variable was freed.
  buf[12] = 42;}
```

Memory Safety - Spatial Error.
tried access to memory outside the bounds of the allocated region.

```
void vulnerable(){
  char buf[12];
  char *ptr = buf[11];
  *ptr++=10;
  // out of allocated memory
  *ptr=42;}
```

CWE 134 - Uncontrolled Format String.
inject format specifiers that can leak memory contents in untrusted input string.
"%4$p":Reads 4th parameter even if doesn't exist
"%6$n":Write to address **pointed to** by 6th parameter

```
int main(int argc, char** argv){
  char buffer[100];
  strncpy(buffer.argv[1]);
  printf(buffer);
  return 0;}
```

Code Injection Attack.
1. Control unchecked variable.
2.Overwrite return address. **3.**Hijack control flow.
4. Arbitrary code execution.

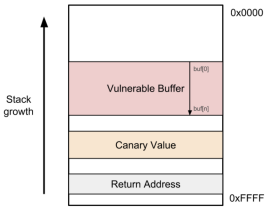
```
void vuln(char* u1){
  // strlen(u1) < MAX?
  char tmp[MAX];
  strcpy(tmp, u1);...
 } vuln(&exploit);
```

Attack Chain:
1. **Memory Safety Violation.** `strcpy` writes past `tmp[MAX]` (spatial error)
2. **Integrity (*C).** Attacker overwrites return address on stack
3. **Location (&C).** Return address now points to shellcode
4. **Usage (*&C).** CPU dereferences corrupted pointer
5. **Code Injection.** Shellcode executes with program privileges

---

Address Space Layout Randomization (ASLR).
Loader/OS randomizes memory regions(code/stack/heap/libraries) at load time. Same code, different addresses on each run. (prevents target addresses prediction/breaks gadgets)
Limitations: 1.Probabilistic defense, not deterministic. 2.Vulnerable to addresses leaks.
3.Some regions remain static *x86*.
4.Performance overhead.(requires the system to keep track of all pointers)

Stack Canaries.
Compiler places random secret value between local buffers and return address. Checked before function returns and abort if corrupted.
(detects buffer overflows before control hijack)



Limitations:
1.Probabilistic, vulnerable to canary leaks.
2.No protection against targeted writes (jumps over canary).
3.No protection against reads.
4.Performance overhead from checks.

Mitigation. A defense mechanism that prevents or reduces the impact of attacks. A mitigation must have three properties:
1. **Effectiveness:** must prevent or defend against attacks
2. **Efficiency:** low comp/memory overhead
3. **Compatibility:** Easy deployment (no major software/hardware changes)

Sandboxing: Isolates process from system resources and other processes. DEP, ASLR, Stack canaries.

Testing. process of executing a program to find errors.(to find bugs/errors and test security properties)
Error. difference between observed behavior and specified behavior (aviolation of underlying spec)
Ideally, we should **test** all (correct control **and** data flow required to trigger bugs):
- **Control-flows.** test all path through the program. (all branches)
- **Data-flow.** test all values used at each location. (all possible values of variables)

Practice is limited by *state explosion* (impossible to test everythig.).

Manual Testing.
designed by human heuristic(intuition).
- **Exhaustive.** cover all input (not feasible)
- **Functional.** cover all requirements (depend on specification)
- **Random.** automate test generation (incomplete)
- **Structural.** cover all code (unit testing)

Automated Testing.
decided algorithmatically
(designed to run program/find bugs/enforce properties).
- **Static analysis.** analyse program without executing it. (imprecise)
- **Symbolic analysis.** converts program to logical equations covering many executions, provides full path coverage(complete). Path explosion: each branch doubles paths ⇒ exponential blowup. Does not scale.
- **Dynamic analysis.** *fuzzing* inspect program by executing it (hard to cover all paths)

Coverage. Testing needs a metric to measure test suite effectiveness. The intuition is that a software flaw can only be detected if the flawed statement is executed. Therefore, the effectiveness of a test suite depends on how many statements are executed.
- **Statement Coverage.** Measures how many statements (e.g., assignments, comparisons, etc.) in the program have been executed by the test suite.
- **Branch Coverage.** Measures how many branches among all possible execution paths have been executed by the test suite.

Fuzzing. random testing technique that mutates input to improve test coverage (scales very well).
State-of-the-art fuzzers use coverage as feedback to guide input mutations.

Types of Fuzzing:
- **Dumb Fuzzing:** unaware of the input structure; randomly mutates input without guidance.
- **Generation-based Fuzzing:** uses model describing valid inputs; generates new input seeds in each round based on model specifications.
- **Mutation-based Fuzzing:** Leverages a set of valid seed inputs; modifies inputs based on feedback from previous rounds. Mutations can be informed by program structure:
  - **White-box** (full access to source code and internal logic)
  - **Grey-box** (partial knowledge, typically coverage feedback only)
  - **Black-box** (no internal knowledge, input/output observation only)

Symbolic Execution with Fuzzing. Whitebox fuzzing uses symbolic execution to generate new seed inputs for unexplored paths when random mutation gets stuck, improving coverage.

Sanitization.
Test cases detect bugs through various mechanisms (enforced policies):
- Assertions
- Segmentation faults
- Division by zero traps
- Uncaught exceptions
- Mitigations triggering termination

cannot detect logic bugs/specification only bugs that violate specific policy **Goal:** Sanitizers enforce policies to detect bugs earlier and increase testing effectiveness.

Address Sanitizer (ASan).
Detects memory errors by placing red zones around objects and checking them on trigger events. It can detect:
- Out-of-bounds accesses to heap, stack, and globals
- Use-after-free
- Use-after-return (configurable)
- Use-after-scope (configurable)
- Double-free and invalid free
- Memory leaks (experimental)

Performance.
adds a 2× slowdown.

Undefined Behavior Sanitizer (UBSan).
Detects undefined behavior by instrumenting code to trap on typical C/C++ undefined behavior:
- Unsigned/misaligned pointers
- Signed integer overflow
- Conversion between floating point types leading to overflow
- Illegal use of NULL pointers
- Illegal pointer arithmetic

Performance.slowdown depends on amount/frequency of checks. only one usable in production.